

智慧图书馆管理系统

项目报告

项目名称:	智慧图书馆管理系统
学号:	1030425120
姓名:	王政南
班级:	计科2501
提交日期:	2026年6月15日
讲解视频网址:	https://www.bilibili.com/video/BV1YtjN67ELB/?share_source=copy_web&vd_source=15598bf84ff1cbf179504bc4b9cfb135

2026 年 6 月 17 日

1 项目概述

本项目实现了一个基于 C++ 的图书管理系统，支持图书与用户的基本管理，以及借阅、归还、查询等操作。系统采用命令行菜单交互方式，用户可以通过输入编号完成相应功能。

1.1 功能简介

系统主要功能包括：

- 添加、删除图书；
- 查询图书信息，支持按编号查询、按书名模糊查询、查看全部图书；
- 添加、删除用户；
- 借阅图书与归还图书；
- 查看某个用户的当前借阅情况；
- 对借阅数量进行限制，每位用户最多借阅 5 本书。

1.2 运行规则说明

- 图书只有在未被借出时才可删除；
- 用户存在未归还图书时不能删除；
- 同一本书同一时间只能被一位用户借阅；
- 借书时会记录借阅日期，归还时会更新历史记录；
- 用户最多同时借阅 5 本图书。

1.3 运行截图

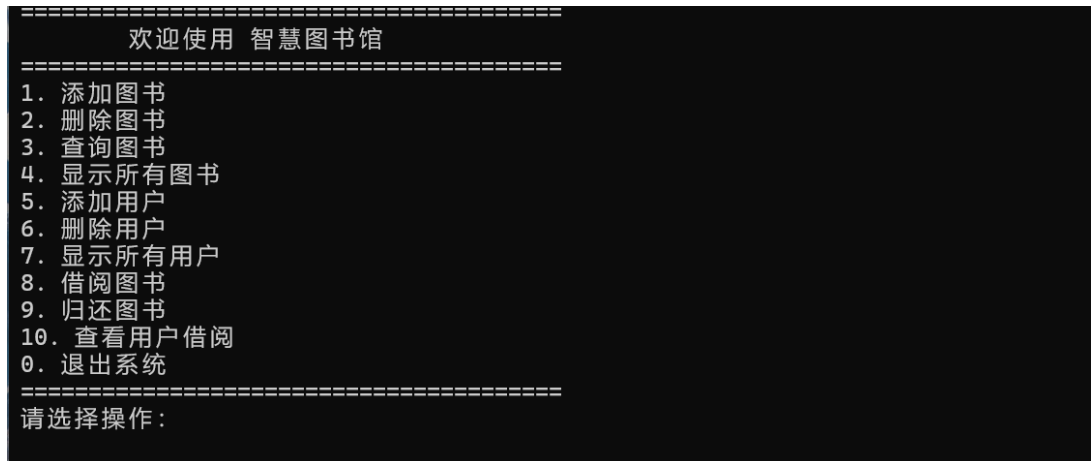


图 1: 系统主菜单界面

2 OOP设计详解

本项目采用面向对象思想进行设计，将图书、用户、借阅记录以及图书馆管理逻辑分别封装到不同的类中，使程序结构清晰、职责明确，便于后续维护和扩展。与此同时，为了体现继承与多态的设计思想，项目新增了抽象基类 `Displayable`，并让 `Book` 和 `User` 继承该基类，实现统一的显示接口。

2.1 类设计与职责划分

1. `Displayable` 类：抽象显示接口

- 作为抽象基类，定义统一的 `display()` 接口；
- 使用纯虚函数强制派生类实现自己的显示方式；
- 体现了面向对象中的多态思想。

2. `Book` 类：图书实体类

- 保存图书编号、书名、作者、ISBN、借阅状态、借阅者编号等信息；
- 继承 `Displayable`，并重写 `display()` 方法；
- 通过 `getter/setter` 控制属性访问，体现封装性。

3. `User` 类：用户实体类

- 保存用户编号、姓名、当前借阅图书列表、借阅历史；

- 继承 Displayable，并重写 display() 方法；
- 提供借书、还书、显示当前借阅信息等功能；
- 限制借阅数量最多为 5 本。

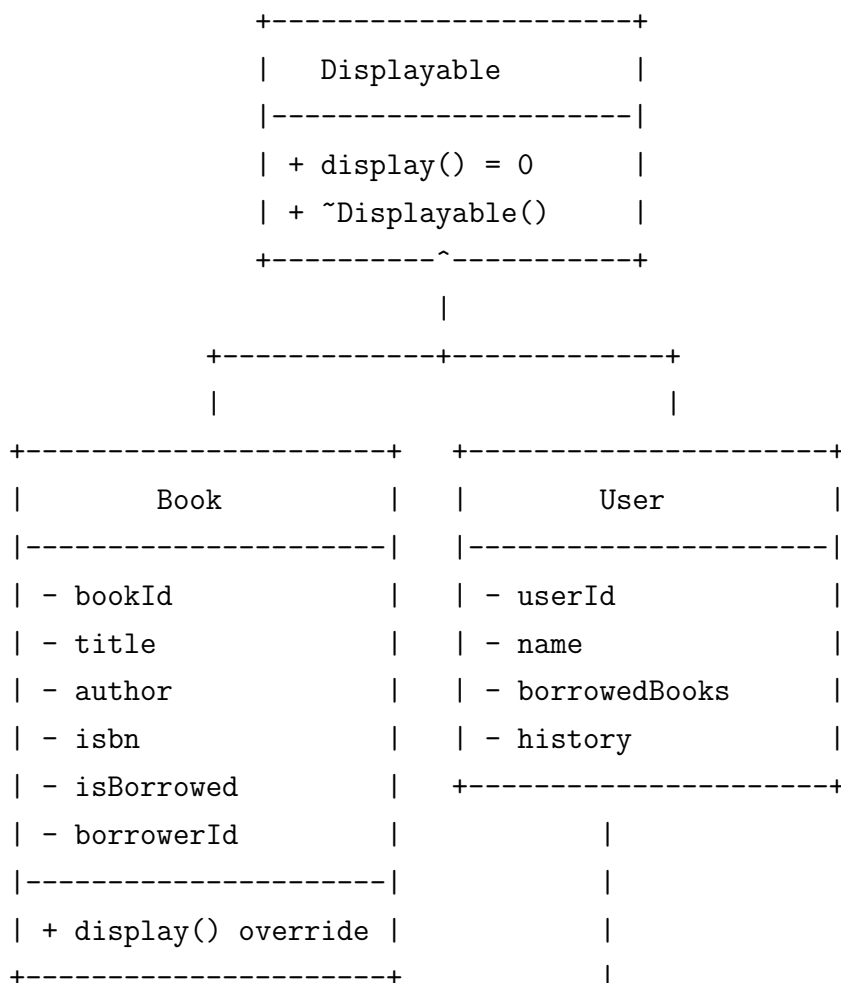
4. BorrowRecord 结构体：借阅记录

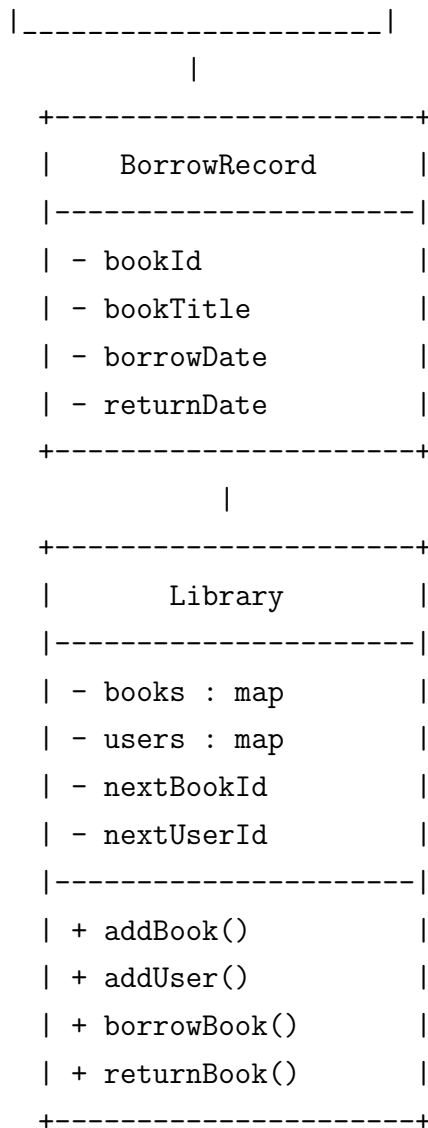
- 用于记录每次借阅的图书编号、图书名、借书日期和还书日期；
- 便于保存借阅历史，增强系统的可追踪性。

5. Library 类：图书馆管理核心类

- 负责管理图书集合和用户集合；
- 提供添加、删除、查询、借阅、归还等菜单功能；
- 负责输入校验和主循环控制，是系统的控制中心。

2.2 类关系图





2.3 为何这样设计

这种设计的优点主要体现在以下几个方面：

- **职责单一：** 每个类只负责自己的事情，避免一个类承担过多功能；
- **数据封装：** 类的成员变量大多为 `private`，外部只能通过接口访问；
- **易于维护：** 若要修改借阅规则，只需要调整 `User` 或 `Library` 的局部逻辑；
- **易于扩展：** 未来可以继续增加“预约功能”“超期罚款”等模块。

2.4 如果不使用 OOP，会怎样

如果不用面向对象，程序很可能会写成一堆全局变量和函数，例如：

- 用全局数组保存图书、用户、借阅记录；

- 所有操作都写成独立函数；
- 数据和逻辑混在一起，函数之间依赖复杂；
- 一旦需求变化，修改成本很高。

与之相比，OOP 的好处是：

- 可维护性更强：修改某个类时影响范围更小；
- 可读性更高：从类名和函数名就能看出功能；
- 更符合现实模型：图书、用户、图书馆本来就是独立对象；
- 便于扩展：新增功能时只需增加新类或新成员函数；
- 可实现多态调用：统一接口处理不同对象，结构更清晰。

2.5 关于继承与多态的说明

本项目通过 `Displayable` 抽象基类、`display()` 纯虚函数以及 `Book`、`User` 的重写，实现了继承与多态的结合。这样做的好处是：系统可以面向父类接口编程，而由不同子类提供各自的显示行为，符合面向对象设计原则。

3 现代C++特性运用分析

本项目中使用了多处现代 C++ 的写法，提升了代码的简洁性、安全性和可读性。下面挑选三处典型代码进行分析。

3.1 使用 vector 管理动态数据

代码中用 `vector` 保存用户当前借阅的图书编号和借阅历史记录：

```
1 vector<int> borrowedBooks;  
2 vector<BorrowRecord> history;
```

优势分析：

- 动态扩容：借阅数量不固定，`vector` 可以自动扩容；
- 使用方便：支持 `push_back`、`erase` 等操作；
- 更安全：避免手动管理数组长度和内存。

如果不使用现代 C++，可能会写成固定长度数组，例如：

```
1 int borrowedBooks[5];  
2 int borrowedCount = 0;
```

这种写法的问题是：

- 容量固定，不够灵活；
- 代码中需要频繁维护下标和计数器；
- 容易越界，安全性较差。

3.2 使用范围 for 遍历集合

例如在显示所有图书时：

```
1 for (const auto& pair : books) {  
2     pair.second.display();  
3 }
```

优势分析：

- 简洁：不需要手动写迭代器初始化和自增；
- 易读：代码更接近自然语言；
- 减少错误：避免迭代器写错、越界等问题。

如果不用范围 for，通常要写成：

```
1 for (map<int, Book>::const_iterator it = books.begin(); it != books.end(); ++it  
    ) {  
2     it->second.display();  
3 }
```

相比之下，现代写法更短、更清晰，也更容易维护。

3.3 使用 auto 简化类型书写

例如：

```
1 auto it = books.find(bookId);
```

优势分析：

- 减少冗长类型：map<int, Book>::iterator 很长；

- 降低书写负担：适合复杂模板类型；
- 更易读：关注变量用途，而不是类型细节。

如果不用 `auto`，则需要写成：

```
1 map<int, Book>::iterator it = books.find(bookId);
```

这种写法虽然也正确，但在复杂模板中会显得冗长，影响代码可读性。

3.4 其他现代写法说明

本项目还用到了以下现代 C++ 风格：

- 使用 `const` 修饰不会修改对象的方法，提高接口安全性；
- 使用初始化列表构造对象，减少重复赋值；
- 使用 `emplace` 直接构造对象，提高效率。

4 核心难点与解决方案

本项目中比较棘手的问题是：如何正确处理借书和还书时的数据一致性。

4.1 问题表现

在最初实现时，可能会出现以下情况：

- 图书已经被借出，但用户记录未同步更新；
- 用户还书后，图书状态没有恢复为“可借”；
- 借阅历史中存在未正确更新的“未归还”记录；
- 用户和图书之间状态不一致，导致系统逻辑错误。

4.2 定位过程

我通过逐步打印输出和检查借书/还书流程，发现问题主要来自两个地方：

- 借书操作只修改了图书状态，没有同步更新用户记录；
- 还书操作中没有严格检查“该书是否确实由该用户借出”。

4.3 解决方案

最终采用了以下处理方式：

- 借书成功后，同时更新图书的借阅状态和用户的借阅列表；
- 还书时先判断借阅者编号是否匹配，确保归还人正确；
- 在用户类中维护借阅历史，归还时更新对应记录；
- 通过统一的流程控制，保证数据同步一致。

4.4 经验总结

这个问题让我认识到，管理系统类项目中最重要的不是界面，而是数据状态的一致性。只要对象之间的信息同步没有做好，就容易出现逻辑错误。因此在设计时必须提前考虑状态流转和边界条件。

5 编译与运行说明

- 操作系统：Windows / Linux 等均可；
- 编译器：支持 C++11 及以上版本的编译器；
- 推荐工具：g++、devc++、Visual Studio 等。

如果在 Windows 下使用 MinGW，可运行：

```
1 g++ -std=c++11 main.cpp -o library.exe
2 library.exe
```

6 总结与心得

通过本次图书管理系统项目，我进一步掌握了 C++ 类、对象、封装、STL 容器以及基本程序设计流程。相比单纯的语法练习，这个项目更接近真实应用场景，让我体会到功能实现与结构设计同样重要。

6.1 项目收获

- 学会了如何使用类来组织程序结构；
- 理解了封装、组合与模块化设计的作用；
- 查阅并熟悉了 vector、map、范围 for、auto 等现代 C++ 写法；

- 提高了对输入校验、异常情况处理和状态同步的认识。

6.2 不足与改进方向

- 目前程序仍是命令行版本，界面较为简单；
- 数据仅保存在内存中，程序退出后会丢失；
- 以后可以加入文件读写，实现持久化存储；
- 还可以扩展预约、超期提醒等功能。

6.3 结语

总体来说，这次项目让我对面向对象程序设计有了更直观的理解，也提升了我独立分析问题和解决问题的能力。后续如果继续完善，我希望把它扩展成一个带有数据持久化和更丰富功能的完整图书管理系统。